ДЛЯ СТАЖЁРА

Разбор результата · неделя 3

https://github.com/OnixxTeam/onix-modulebank

Проверенный commit: 1d970c8eb028fafc9b75336b221df0da281db0c4

Итог

Незачёт. Подтверждено нарушение обязательного условия учебного результата.

Сохраняются подтверждённые блокирующие условия: Предметное состояние и неизменяемая история согласованы. Другие технические отклонения выделены отдельно и сами по себе не являются причиной незачёта.

Условия учебного зачёта

Воспроизводимый запуск по документированным входам: Подтверждено. Чистая сборка и запуск воспроизведены на закреплённом commit с документированными проверочными входами.

C# generic runtime, PostgreSQL state и самостоятельный Python-периметр: Подтверждено. C# generic runtime и обе карты используют PostgreSQL state; Python-периметр собран из исходников и отделён от предметных решений.

Платёжные success/reject и автоматическая/ручная ветви завершаются: Подтверждено. Сохранённые проверки подтверждают платёжные success/reject и автоматическую/ручную ветви второго процесса.

Повтор и конкуренция не создают второй бизнес-эффект: Подтверждено. Конкурентные submit/manual и повторы доставки сохраняют один предметный эффект; несовместимые ответы выделены отдельно.

Финальный результат подтверждён связанной квитанцией или решением: Подтверждено. Проверенные предметные финалы связаны с сохранёнными квитанциями или решениями; конкретные обходы рассматриваются отдельно.

Недоверенный клиент не назначает успех и не обходит полномочия/подпись: Подтверждено. Подпись и полномочия ordinary API-клиента проверены; замечания к формату входа не подменяют доказательство обхода доверия.

Предметное состояние и неизменяемая история согласованы: Требует исправления. Runtime подтверждает terminal state без финальной записи operation_events. Это пропущенная история, а не неверный формат существующего события.

Outbox/Inbox не теряют сообщения при обязательных сбоях и рестартах: Подтверждено. Durable Outbox/Inbox, lost response, ранняя доставка и recreate подтверждены. Применение сохранённого Inbox reconciler не считается потерей сообщения.

Исправление замечания проверяющего

Правила учебного зачёта пересмотрены: частное техническое несоответствие больше не означает автоматический незачёт. Замечания и прежние доказательства сохранены; каждое замечание классифицировано по конкретному последствию.

Готовность сдачи

Комплектность и commit: Exact commit доступен, дерево чистое; 342 tracked files содержат C#, SQL, Python, contracts, tests и build inputs.

Холодная сборка: Trusted cold build собрал exact commit без изменения исходников участника.

Первый запуск: Новый isolated project/volume достиг readiness и выполнил полный public runtime без ручной настройки исходников или базы.

README и конфигурация: README подробно описывает архитектуру, конфигурацию, migrations, workflow, Python-периметр, provider, запуск и диагностику.

Команда проверки: README называет ./check.sh открытой black-box проверкой текущего задания, но tracked wrapper запускает week-2-public-check/1.1 и создаёт week-2-public-report.json.

Что проверено

Проверены exact commit, isolated cold build/start, 55 public checks, provider success/reject/lost-response/early/transient, Outbox stop/resume, duplicate/conflict, reconciler recreate, 20-way submit/manual, malformed callback matrix, exact signed body, финальная история operation, 105 Python tests, 92 C# unit tests, DB roles/projections, source/history secret scan и active SQL/C#/Python paths.

Сильные стороны

1. Все 55 public checks прошли без изменения исходников сдачи.

2. Concurrent submit обоих operation kinds создаёт один pinned process; mixed manual decisions сохраняют один audited effect.

3. Success/reject, lost-response, early callback, transient retry и reconciler recreate подтверждены авторитетными SQL/provider evidence.

4. Integration roles ограничены fixed functions без table DML; Python services используют один локальный image и adapter не имеет DB credentials.

5. В sandbox прошли 105 Python tests и 92 C# unit tests; собственный DB integration suite покрывает claim/reclaim, fencing, Outbox/Inbox и manual branches.

6. Исправлено прежнее замечание о повторном action bootstrap: action operational state больше не активируется безусловно.

Работа с предыдущими замечаниями

Исправлено: Bootstrap больше не вызывает action activate после idempotent publish и сохраняет operator disable/default state.

Сохранилось / частично исправлено: Generic idempotency key по-прежнему не включает executed action version и cached envelope не закрепляет original action version.

Сохранилось / частично исправлено: publish_flow по-прежнему читает отсутствие версии до обычного INSERT без route/version lock или conflict recovery.

Сохранилось / частично исправлено: SetAtPointer по-прежнему заменяет любой промежуточный non-object новым JsonObject и не умеет адресовать существующий JsonArray.

Сохранилось / частично исправлено: Claim всё ещё вычисляет lease только из общего profile value; test lease 2s короче допустимого task timeout 5s, renewal отсутствует.

Совпадение дефекта в двух версиях отделено от факта доставки обратной связи. Доказательств доставки прежних отчётов до этой сдачи в доступных материалах недостаточно; усиление за подтверждённый повтор не применяется.

Замечания и способы исправления

Высокий приоритет: Terminal status не имеет immutable operation event

Влияние на зачёт: блокирует. Runtime подтверждает terminal state без финальной записи operation_events. Это пропущенная история, а не неверный формат существующего события.

Наблюдение: payment.approve_v1 и payment.reject_v1 обновляют core.operations до COMPLETED/REJECTED, но не вставляют запись в core.operation_events. Runtime показал две COMPLETED approval operations, у которых остались только OPERATION_CREATED и OPERATION_SUBMITTED.

Последствие: Авторитетное состояние и audit history расходятся; результат невозможно восстановить по operation events, поэтому применяется стоп-фактор состояния/события.

Что изменить: В той же PostgreSQL transaction и под lock сохранять ровно один OPERATION_COMPLETED/OPERATION_REJECTED с привязкой к persisted receipt/decision; запретить повторный terminal transition.

Как проверить: Для automatic и manual approval, provider success/reject проверить ровно один соответствующий final event; rollback/fenced failure не должны менять ни status, ни history.

Где смотреть: db/migrations/033_payment_approve.sql:17-38; db/migrations/034_payment_reject.sql:14-35

Высокий приоритет: Generic replay обходит exact-byte conflict

Влияние на зачёт: не блокирует, замечание сохраняется. Byte-only replay не создаёт нового эффекта и не меняет содержание квитанции; нарушение exact bytes остаётся техническим.

Наблюдение: api.invoke хеширует нормализованный p_payload::text и возвращает cached envelope до receipt.accept_v1. Два корректно подписанных semantic-equivalent тела с разными exact bytes и одним messageId/Idempotency-Key получили HTTP 200/200 с исходным correlationId.

Последствие: Изменение подписанных bytes не обнаруживается, хотя Inbox хранит exact raw-body hash; конфликтная доставка маскируется как успешный duplicate replay.

Что изменить: Для signed receipt включить trusted rawBodyHash в generic idempotency conflict identity либо выполнять exact-byte/signature-specific check до cached replay.

Как проверить: Отправить compact signed receipt, затем корректно переподписанный body с иными whitespace/key order и тем же key/messageId: второй запрос должен вернуть 409, сохранив одну Inbox/receipt запись.

Где смотреть: src/Course.Api/Endpoints/ActionEndpoint.cs:103-144; db/migrations/018_replay_returns_the_original_envelope.sql:80-118; db/migrations/038_receipt_accept_and_inbox.sql:49-109

Средний приоритет: Callback validation допускает malformed values

Влияние на зачёт: не блокирует, замечание сохраняется. Malformed callbacks дали неконтрактные ошибки, но сохранённые probes не установили Inbox mutation; классификация HTTP не равна ложному финалу.

Наблюдение: Regex с $ принимает финальный LF/CRLF, occurredAt проверяется только по форме и принимает невозможную календарную дату, а membership test для object result выбрасывает TypeError. В результате malformed callbacks дали 500 либо дошли до платформы и вернули 422/503 вместо adapter HTTP 400.

Последствие: Недоверенный legacy callback пересекает anti-corruption boundary или вызывает внутреннюю ошибку; provider получает нестабильную классификацию и может повторять непригодное сообщение.

Что изменить: Сначала проверять тип каждого поля, использовать fullmatch/\Z для строк без CR/LF и RFC3339 parser с проверкой реальной даты, сохраняя исходную occurredAt строку после

validation.

Как проверить: Добавить unit/HTTP cases для object/array/null во всех полях, невозможных дат и CR/LF в начале/середине/конце: каждый должен вернуть 400 без upstream call и Inbox mutation.

Где смотреть: python/src/perimeter/callback.py:24-72; python/src/perimeter/adapter.py:60-87; python/tests/test_callback.py:71-107

Средний приоритет: Outbox test profile использует неверный retry budget и lease

Влияние на зачёт: не блокирует, замечание сохраняется. Lease/retry budget не совпадает с test profile, но текущие lost-response/recreate сценарии сохраняют один платёж и durable сообщения.

Наблюдение: retry_policy задаёт пять attempts и пять delay values, а claim_outbox всегда выдаёт 30-second lease. Опубликованный test profile требует четыре attempts, 200/400/800 ms и lease 2 seconds.

Последствие: Сбойный provider получает лишний запрос, а crash recovery dispatcher может занимать до 30 секунд вместо проверочного бюджета.

Что изменить: Загружать profile-aware четыре total attempts с тремя delays и рассчитывать lease из test/production configuration без literals в claim function.

Как проверить: На persistent transport failure получить DEAD ровно после четырёх claims с 200/400/800 ms; после crash владельца delivery должна reclaimed примерно через 2 seconds в test profile.

Где смотреть: db/migrations/039_delivery_functions_and_perimeter_roles.sql:13-20; db/migrations/039_delivery_functions_and_perimeter_roles.sql:40-70; db/migrations/039_delivery_functions_and_perimeter_roles.sql:131-193

Средний приоритет: Bootstrap безусловно возвращает bundled flow v1 в active state

Влияние на зачёт: не блокирует, замечание сохраняется. Bootstrap меняет active version для новых процессов, не переписывая pinned старые. Это дефект операционного управления версиями; не доказано изменение исполненного результата.

Наблюдение: Action bootstrap исправлен и только публикует manifests, но для payment-processing/payment-review он после каждого publish всегда вызывает ActivateAsync(v1), исходя из предположения, что других версий не будет.

Последствие: После публикации и активации flow v2 очередной compose bootstrap молча вернёт default на bundled v1, перезаписав operator-controlled operational state.

Что изменить: Активировать bundled flow только при first install/отсутствии active version; повторный bootstrap должен сохранять выбранную оператором active version.

Как проверить: Опубликовать и активировать flow v2, повторить bootstrap и убедиться, что active остаётся v2; fresh volume по-прежнему должен автоматически активировать v1.

Где смотреть: src/Course.Cli/Commands/BootstrapCommand.cs:88-120

Средний приоритет: Локальная команда проверки относится к Week 2

Влияние на зачёт: не блокирует, замечание сохраняется. Локальная команда проверяет прошлую неделю; текущий предметный runtime проверен отдельно и замечание остаётся в диагностике.

Наблюдение: README объявляет ./check.sh открытой black-box проверкой текущей сдачи, но wrapper запускает tracked public_check.py с toolVersion week-2-public-check/1.1 и пишет week-2-public-report.json.

Последствие: Разработчик получает зелёный отчёт прежней недели, не проверяя provider, Python services, Outbox/Inbox, signed receipt и manual process текущего инкремента.

Что изменить: Заменить wrapper на pinned Week 3 checker либо самодостаточный эквивалент и формировать week-3-public-report.json.

Как проверить: На clean clone одна документированная команда должна поднять десять services, проверить Week 3 contracts и выдать report с week-3 manifest/tool version.

Где смотреть: README.md:251-270; autocheck/check.sh:16-20; autocheck/public_check.py:29-30

Порядок доработки

1. Сначала добавить атомарные final operation events и regressions для automatic/manual/provider terminal paths.

2. Перенести exact raw-body conflict перед generic cached replay и добавить differently-serialized signed-body test.

3. Ужесточить callback type/date/CRLF validation до любого upstream вызова.

4. Привести Outbox test profile к четырём attempts/2-second lease и перестать активировать bundled flow поверх operator choice.

5. Заменить Week 2 wrapper воспроизводимой pinned Week 3 командой проверки.

Границы проверки

Это пересчёт по обновлённой калибровке с дополнительной проверкой документации и истории замечаний. Использованы прежние runtime evidence того же commit; новый runtime-run не заявляется.

Проверен только указанный commit; исходники участника не изменялись.

Полный C# integration suite не запускался в sandbox без Docker socket; прошли 92 C# unit tests, 105 Python tests и независимый trusted runtime на реальных контейнерах.

Gitleaks findings текущей истории относятся к документированным development/test fixtures; пригодность этих значений вне локального контура не установлена.

Доставка прежних PDF не подтверждена; repeat-cap не применялся.

Новая проверка исполнения не выполнялась. Пересмотрены условия зачёта и классификация сохранённых наблюдений того же commit; прежняя оценка и архив сохранены.